

Preserving an Artifact First Evidence Architecture for StratLake Trade Engine

Executive summary

The StratLake Trade Engine repository already encodes the right **artifact-first** principle for Milestone 36. Its README, M36 release notes, catalog design docs, and validation checklist repeatedly state that **canonical artifacts remain the source of truth**, while the SQLite catalog index is optional and disposable, lineage exports are local derived views, dataset and feature lineage is explicit and optional, and the system intentionally avoids introducing a second registry, graph store, backend service, or inferred-lineage layer. In other words, the architectural intent is already strong. [1](#) [2](#) [3](#)

The main Milestone 37 challenge is therefore **not conceptual drift in the repository's stated architecture**. It is the risk that convenience surfaces introduced in M36 — especially the **derived SQLite index, portable lineage exports, and serialized lineage metadata carried in derived payloads** — become treated by humans, downstream tools, notebooks, wrappers, or future automation as if they were canonical. That would turn read models into governing models. The repo currently mitigates some of that risk through direct-scan defaulting, staleness checks, parity tests, explicit rebuild failures, portability validators, and “no mutation” boundaries, but the boundary is still more **policy-by-convention** than **policy-by-contract** at some interfaces. [4](#) [5](#) [6](#) [7](#) [8](#)

My bottom-line recommendation is to make canonicity a **first-class, machine-checkable contract** across code, storage, UI, and governance. Specifically, M37 should add: a required **canonicity envelope** on every derived artifact; visibly separate storage namespaces for canonical vs. derived outputs; stronger CLI/UI “derived, non-authoritative” cues; architecture tests that forbid derived surfaces from feeding write/promotion/governance decisions; export and index fingerprints that point back to canonical source files; and migration shims that preserve M36 compatibility while making it harder for derived views to become de facto source-of-truth. Those changes align with both the repository's existing M29–M36 philosophy and broader provenance guidance: provenance is valuable because it helps establish trust and accountability, but it is still a **representation of derivation**, not the thing itself. [9](#) [10](#) [11](#)

Current repository state and the M36 artifact first inventory

The repository's M29 foundation defines the catalog as a **read-only observer** over existing artifact roots, registries, manifests, markers, summaries, and validation bundles. The design explicitly says that if a persisted cache or export is ever added, it must remain a **derived convenience artifact** and must not become canonical for run identity, lifecycle status, lineage, or artifact inventory. M35 extends that boundary to evidence bundles without turning the catalog into a database, cache, search backend, or policy engine. M36 then adds performance and interoperability features while restating that source artifacts remain canonical. [9](#) [10](#) [2](#)

The most important M36 design and implementation assets for an M37 “preserve artifact-first” effort are these:

Category	Repository assets	Why they matter
Core architecture statements	<code>README.md</code> ¹ , <code>docs/m36_release_notes.md</code> ² , <code>docs/m36_release_validation_checklist.md</code> ³	State the non-goals and the intended canonical boundary.
Foundational catalog design	<code>docs/milestone_29_unified_research_catalog_design.md</code> ⁹ , <code>docs/catalog_indexer.md</code> ¹² , <code>docs/catalog_lineage.md</code> ¹³ , <code>docs/catalog_query.md</code> ¹⁴	Define source-of-truth inputs, catalog semantics, and explicit-lineage rules.
Public catalog API surface	<code>src/catalog/__init__.py</code> ¹⁵	Shows that direct scan, derived index, lineage export, notebook helpers, and workflow helpers are all part of one public surface.
Data models	<code>src/catalog/models.py</code> ¹⁶	Defines <code>CatalogRecord</code> , <code>ArtifactRecord</code> , <code>LineageEdge</code> , and validation structures.
Derived read model	<code>src/catalog/derived_index.py</code> ⁴ , <code>src/cli/catalog_index.py</code> ¹⁷	Implements the optional SQLite index, index metadata, validation, and load modes.
Exported lineage views	<code>src/catalog/lineage_export.py</code> ⁵ , <code>src/cli/export_catalog_lineage.py</code> ¹⁸	Implements deterministic OpenLineage-style and PROV-style local JSON exports.

Category	Repository assets	Why they matter
Explicit lineage metadata contract	<code>src/catalog/lineage_fingerprints.py</code> ¹⁹ , <code>src/data/feature_metadata.py</code> ²⁰	Produces deterministic dataset/feature lineage blocks and fingerprints when explicit inputs exist.
Shared workflow surfaces	<code>src/catalog/workflows.py</code> ⁶ , <code>src/cli/query_catalog.py</code> ²¹ , <code>src/cli/explore_catalog_evidence.py</code> ²²	Make direct, index, and auto modes available across CLI, notebooks, and wrappers.
Validation and regression harness	<code>tests/test_m36_deterministic_validation.py</code> ⁷ , <code>tests/test_catalog_lineage_export.py</code> ²³ , <code>tests/test_dataset_feature_lineage.py</code> ²⁴ , <code>tests/test_catalog_cli_api_ergonomics.py</code> ⁸ , <code>tests/catalog_scale_fixtures.py</code> ²⁵	Prove parity, disposability, portability, explicit-edge preservation, and non-mutation.
Release and CI hardening	<code>.github/workflows/milestone_validation.yml</code> ²⁶ , <code>.github/workflows/release.yml</code> ²⁷ , <code>.github/workflows/ci.yml</code> ²⁸	Lock in validation, release, SHA pinning, and cross-platform checks relevant to preserving boundaries.

The current M36 code patterns are unusually disciplined for an artifact-first system. The catalog indexer reads known artifact roots and registry paths only, computes deterministic IDs, preserves repository-relative portable paths, and carries forward explicit lineage metadata when present rather than inventing it. The lineage layer derives edges only from explicit metadata fields and source artifacts and skips unresolved references instead of guessing. The derived index stores its own metadata fields such as `is_derived: true` and `canonical_source: "artifacts"`, and validation compares fingerprints against a fresh direct scan. The export layer validates closed graphs, forbids absolute paths and URI-like strings, and keeps original StratLake edge types even when rendering OpenLineage-style or PROV-style views. ¹² ¹³ ⁴

⁵ ²⁹

From a provenance perspective, that is a sound baseline. W3C PROV defines provenance as information about the entities, activities, and people involved in producing a thing, intended to support assessments of quality, reliability, and trustworthiness; it is a model for **interchangeable provenance information**, not a replacement for the source object itself. Microsoft's CQRS guidance makes the same distinction operationally: the event/write store is the single source of truth, while read models and materialized views are optimized query projections that may be stale and can be rebuilt. ³⁰

How artifacts and derived surfaces work today

Today's Canonical-to-Derived flow in StratLake is coherent. The canonical layer consists of the artifact tree and related provenance inputs: marker files, registries, manifests, metrics and summaries, checkpoint/ scenario files, governance bundles, milestone/release validation bundles, and the artifact directory structure itself. Those are the inputs that M29 explicitly labels as source-of-truth. M36 preserves that. 9

12 2

The runtime flow can be summarized as follows:

Surface	Created by	Stored as	Queried through
Canonical artifacts	Execution workflows already writing manifests, marker files, registries, metrics, summaries, QA, robustness, governance, milestone and release validation outputs	JSON/JSONL/CSV plus directory roots under <code>artifacts/</code>	<code>build_catalog()</code> direct s
Derived index	<code>build_derived_index()</code> and <code>python -m src.cli.catalog_index build</code>	SQLite with <code>metadata</code> and <code>catalog_records</code> tables	<code>load_catalog_records(. mode="index" "auto")</code> af validation
Lineage exports	<code>export_lineage*()</code> and <code>python -m src.cli.export_catalog_lineage</code>	JSON, either <code>openlineage</code> or <code>prov</code> format	Read as exported files, not re ingested as canonical catalog state
Dataset/ feature lineage metadata	<code>build_feature_metadata_summary()</code> plus any manifests/summaries/registry records already carrying explicit blocks	Embedded dicts on source artifacts and then on <code>CatalogRecord.metadata</code>	Propagated by catalog/index export layers
Evidence views	<code>build_evidence_explorer_view()</code> or <code>build_evidence_view_for_workflow()</code>	In-memory view or optional rendered output file	Direct scan or validated index

A few implementation details are especially important for M37. First, the index metadata already records that it is derived, rebuildable, and rooted in canonical artifacts; it also stores a source fingerprint and rejects stale or incompatible indexes rather than trusting them. Second, `auto` mode uses an index only when it is present and valid; stale or mismatched indexes raise rebuild errors. Third, lineage export intentionally operates over explicit `CatalogRecord` and `LineageEdge` objects and keeps a one-hop boundary for

selected-run exports. Fourth, dataset and feature lineage metadata is only preserved when explicitly present; the system does not infer new dataset graph edges from those blocks. 4 5 19 29

The tests confirm those promises. The deterministic M36 stack test asserts parity among direct, index, auto, and helper loads; verifies index disposability and rebuildability; checks that exports preserve edge counts and lineage metadata without adding new edges; verifies that source artifacts are not mutated; and checks for portability by rejecting machine-specific paths. The scale fixture builds a 48-record synthetic tree spanning strategies, alphas, portfolios, campaign/scenario cases, robustness bundles, governance bundles, milestone validation bundles, release validation artifacts, and registry-only records. 7 23 24 8 25

That is a strong starting point. But it is still possible for derived surfaces to become *operationally canonical* even when they are not *architecturally canonical*. Provenance research repeatedly warns that provenance is essential for accountability, reproducibility, and audit, yet must be purpose-fit and carefully represented because it can become large, partial, or system-specific. Interoperability-oriented packaging work such as Workflow Run RO-Crate also shows that once provenance becomes an exchange surface, it starts acquiring its own consumers and compatibility expectations. That is exactly where M37 should focus. 32

Failure modes that could let derived outputs become canonical

The clearest current risk is **semantic flattening**: the derived index stores serialized `CatalogRecord` payloads that look complete enough for many downstream tasks, and the export layer renders nodes and relationships with preserved facets. That completeness is good for interoperability and speed, but it also makes derived payloads *feel* authoritative. A future consumer that never reopens `manifest.json`, `summary.json`, registry rows, or marker files could accidentally turn “serialized observation of source artifacts” into “source artifact.” 16 4 5

A second risk is **path and namespace ambiguity**. Canonical artifacts and derived outputs both live under the broad `artifacts/` tree in documented examples. For humans, `artifacts/catalog_index/catalog_index.sqlite` and `artifacts/lineage/openlineage.json` can look like first-class artifact products rather than caches or views. For scripts, a filesystem walk that is insufficiently scoped can begin to ingest derived outputs as if they were peer evidence. The current indexer only scans known families and indicator files, which reduces this risk, but storage naming still invites misinterpretation. 2 12 3

A third risk is **auto-mode forgetfulness**. `direct` remains the default, which is good, but once wrappers or notebooks standardize on `auto`, users may stop noticing whether a result came from canonical direct scan or a validated derived index. Microsoft’s CQRS guidance is relevant here: read models are useful precisely because they are optimized projections, but that same convenience means teams must be careful that query models do not quietly inherit write-model authority. 6 21 33

A fourth risk is **future write-path contamination**. Today’s repository carefully keeps read-only boundaries. But because `src/catalog/__init__.py` exports direct scan, derived index, exports, notebook helpers, and workflow helpers in one public package, a future milestone could accidentally use a derived surface inside a workflow that emits governance, promotion, or release decisions. That would violate the repository’s own M29/M35/M36 design boundary even if the code path were technically “read-only.” 15

9 10

A fifth risk is **schema capture**. Once the OpenLineage-style / PROV-style JSON files are consumed by external tooling, those files can become long-lived integration contracts. If external users stop treating them as “local standards-oriented JSON views” and start treating them as the system’s authoritative lineage interface, the interchange format will begin to constrain the internal canonical model. That is not hypothetical; provenance interoperability work such as W3C PROV and Workflow Run RO-Crate exists precisely because lineage consumers want portable semantic contracts. That makes contract discipline more important, not less. 2 5 34

A sixth risk is **metadata overreach through convenience**. Dataset and feature lineage blocks are intentionally explicit and optional in M36, and the current indexer correctly preserves but does not infer them. Even so, once those blocks are easily available on derived index rows and export facets, downstream consumers may begin reasoning from those blocks without checking whether the source artifact actually declared them, whether the block came from registry vs. summary vs. manifest precedence, or whether an older artifact legitimately lacks them. Provenance literature consistently notes that provenance capture is useful but must be tailored and interpreted in context; otherwise, captured metadata can look more complete than it really is. 20 29 24 35

Recommended M37 design changes

The best way to preserve artifact-first architecture is to apply the same rule everywhere: **every derived surface must identify, defer to, and be invalidated by its canonical source**. The table below summarizes the most important changes.

Concern	Current M36 behavior	Recommended M37 behavior
Canonicity metadata	Index metadata says <code>is_derived</code> and <code>canonical_source: artifacts</code> ; exports do not carry an equally strong canonicity envelope	Add a required <code>canonicity</code> block to every derived artifact with <code>authority_kind</code> , <code>authority_paths</code> , <code>authority_fingerprint</code> , <code>derived_class</code> , <code>rebuildable</code> , <code>non_authoritative</code> , <code>write_back_forbidden</code> , and <code>stale_if_source_changes</code>
Storage layout	Derived outputs often live under general <code>artifacts/</code> examples	Move or alias all M37 generated derived outputs under a clearly marked namespace such as <code>artifacts/_derived/</code> or <code>artifacts/cache/</code> , while keeping compat paths temporarily
Load-path transparency	<code>direct</code> is default; <code>auto</code> can hide when index is being used	Emit explicit load provenance on every CLI/view result: <code>loaded_from=direct index</code> , <code>canonical_source=artifacts</code> , <code>index_validated_at_load=true false</code>
Export contract	Exports preserve edge types and paths, but do not force canonical source resolution	Add canonical source references and source fingerprints to export headers and node facets; keep current payload fields for compatibility

Concern	Current M36 behavior	Recommended M37 behavior
Governance boundary	Read-only by design, but not yet enforced as an architectural invariant	Add architecture tests forbidding derived index / export modules from being imported into writer, registry, promotion, or governance-decision paths
Human interpretation	Docs say “derived”; UI/CLI output is less forceful	Add visible “Derived / Rebuildable / Not source of truth” banners and “Open canonical manifest/summary” affordances everywhere derived outputs are shown
Access control	No differentiated authorization model is expressed in M36 docs	Split permissions: canonical artifact writers vs. derived rebuilders vs. read-only consumers

These changes are consistent with external best practice. W3C PROV frames provenance as a representation of how something was produced, supporting trust and interoperability; it does not collapse the distinction between the thing and its provenance description. Microsoft’s CQRS guidance is even more direct: materialized read models are optimized query surfaces, and when combined with event sourcing, the event/write store remains the single source of truth while views are regenerated snapshots or caches. ³⁰

The most important concrete M37 technical recommendation is a **Canonicity Envelope v1**. This should be attached to every derived index metadata record, every lineage export root object, and every rendered evidence view header. A representative schema would look like this:

```
{
  "canonicity": {
    "authority_kind": "artifact_tree",
    "authority_root": "artifacts",
    "authority_paths": [
      "artifacts/strategies/strategy_000/manifest.json",
      "artifacts/strategies/registry.jsonl"
    ],
    "authority_fingerprint": "<sha256 over canonical source set>",
    "derived_class": "sqlite_read_model",
    "rebuildable": true,
    "non_authoritative": true,
    "write_back_forbidden": true,
    "stale_if_source_changes": true,
    "resolver_hint": "reopen canonical manifests/registries for decisions"
  }
}
```

That change would convert today’s architectural intent into a machine-readable contract. It also allows M37 to reject or warn on any consumer that tries to use a derived object in a decision path without an explicit unwrap-to-canonical step. The repo already has the raw ingredients for this: relative source paths, source files, source fingerprints, and explicit validation layers. ^{4 5 31 29}

The next recommendation is a **Resolver-first API rule**. Leave `build_catalog()` as the canonical loader. Keep `load_catalog_records()` for read-mode acceleration, but add a small resolver API that any policy, governance, or write-adjacent workflow must call before using record fields as authoritative. For example, `resolve_canonical_record(record_or_row)` should reopen the artifact root and canonical source files and return a provenance-checked authoritative snapshot. This preserves M36 performance gains for discovery while preventing policy code from acting on serialized projections. That is directly analogous to CQRS's separation between read projections and authoritative write models. 4 6 33

I also recommend **stronger UI and UX cues**. Every CLI and notebook-rendered output that was loaded from an index or rendered from exported lineage should include a visible header such as: **Derived view — validated against canonical artifacts — do not edit or cite as authoritative artifact state**. Explorer and notebook helpers should add direct links or fields for `source_manifest_path`, `source_registry_path`, and `source_marker_path`, because the safest way to prevent canonicalization is to make the canonical path more visible than the convenient view. The current query and explorer surfaces already carry enough source-path information to support this. 21 22 16

On governance and access control, M37 should formalize three classes of actors: **canonical artifact writers, derived rebuilders, and read-only consumers**. Canonical artifact roots should only be writable by workflow roles that already produce them. Derived namespaces should be writable by indexing/export tooling. Consumers should not have permission to mutate either. If role-based filesystem or object-store policies are available later, that separation should be enforced there too; if not, it should at least be encoded in repository policy and test fixtures. This is especially important because provenance research emphasizes accountability and audit utility, which only helps if the custody chain around canonical evidence stays clear.

36

Finally, M37 should add **architecture and contract tests** beyond the existing parity tests. The current tests prove direct/index/export parity and non-mutation. M37 should add tests that fail if: a module in a writer/governance/promotion path imports `derived_index` or `lineage_export`; a derived artifact lacks a canonicity envelope; a derived consumer path omits “loaded_from” signaling; or a documentation example presents an index/export path without clarifying its non-authoritative status. These tests should complement the existing M36 deterministic suite, not replace it. 7 8 3

Migration strategy and prioritized action plan

The good news is that this can be done **additively** and **without breaking old artifacts**. M36 already preserved backward compatibility by keeping lineage metadata optional and explicit, keeping direct scan canonical, and treating stale or incompatible indexes as rebuild failures instead of silent fallbacks. M37 should preserve the same posture. 2 4 24

A practical migration path is:

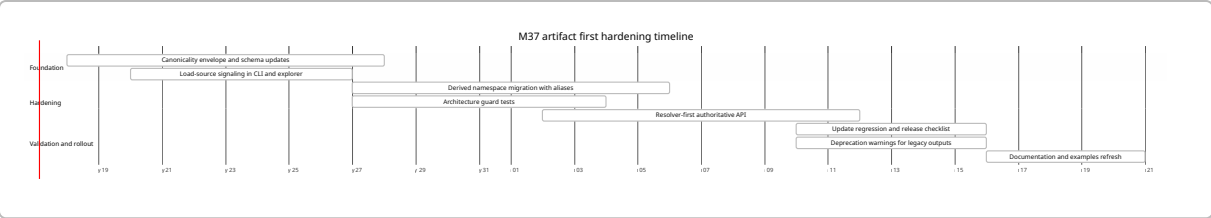
Phase	What changes	Compatibility posture
Immediate	Add <code>canonicity</code> envelopes and load-source signaling to new indexes, exports, and explorer/query outputs	Backward-compatible readers accept old payloads but warn that they are “legacy derived artifacts without canonicity envelope”
Near term	Introduce <code>artifacts/_derived/</code> and keep old output paths as deprecated aliases	Old scripts continue to run; docs shift to new namespace first
Medium term	Add resolver-first APIs for policy/governance/write-adjacent consumers	Existing discovery consumers unchanged; authoritative consumers opt in first, then become mandatory
Later	Make architecture tests and policy checks block unsafe imports and unsafe derived-only decision paths	Hardening only; does not alter canonical artifact schemas

The recommended action plan is below.

Priority	Recommendation	Why it should come first	Effort	Delivery risk	Risk reduction
High	Add <code>canonicity</code> envelope to all derived artifacts	Converts convention into machine-readable contract	Medium	Low	High
High	Add visible load-source and “derived/non-authoritative” cues to CLI, notebook, explorer, and export headers	Prevents human/operator confusion immediately	Small	Low	High
High	Add architecture tests forbidding derived surfaces in writer/governance/promotion code paths	Prevents future accidental canonicalization by code reuse	Medium	Medium	High
High	Move derived outputs to <code>artifacts/_derived/</code> with compat aliases	Reduces namespace confusion and accidental ingestion	Medium	Medium	High
Medium	Add resolver-first authoritative APIs for decision-making code	Forces unwrap-to-canonical before sensitive use	Medium	Medium	High

Priority	Recommendation	Why it should come first	Effort	Delivery risk	Risk reduction
Medium	Add export/source fingerprint references to every node/header	Makes stale external copies easier to detect	Medium	Low	Medium
Medium	Formalize access-control and role policy in docs and validation fixtures	Hardens governance and operational clarity	Small	Low	Medium
Lower	Add optional attestations/signatures for derived artifacts	Useful for supply-chain style trust, but not necessary to solve canonicalization first	Medium	Medium	Medium

A reasonable implementation timeline is:



This plan is intentionally conservative. Provenance and read-model literature both suggest that the moment a derivative surface becomes widely consumed, its stability and semantics matter more. M37 should therefore tighten semantics **before** more external consumers accumulate around M36 exports and indexes.

37

Open questions and sources

The highest-confidence conclusion from the available evidence is that StratLake’s current repository already *wants* the right answer: artifacts first, derived surfaces second. M37 should therefore focus on **enforcement and signaling**, not on changing the core philosophy. The biggest unresolved design choice is whether to keep derived outputs under the `artifacts/` tree with a clearer sub-namespace, or move them entirely outside that tree. The repository evidence supports either approach; my preference is a visibly marked sub-namespace first because it minimizes migration pain while still clarifying semantics. 2 3

I was able to ground the repository analysis in the connected GitHub repo and corroborate the broader design recommendations with official W3C provenance specifications, Microsoft’s CQRS/event-sourcing guidance, and original academic provenance literature. I was not able to retrieve a directly citable official OpenLineage specification page through the web tool during this session, so discussion of “OpenLineage-style” exports is grounded primarily in the repository’s own implementation and documentation rather than an external OpenLineage source. 2 5

Primary sources used, with the repository listed first:

Source	Role in this report
StratLake Trade Engine repository docs, code, tests, and workflows — especially <code>README.md</code>, <code>docs/m36_release_notes.md</code>, <code>docs/m36_release_validation_checklist.md</code>, <code>docs/milestone_29_unified_research_catalog_design.md</code>, <code>docs/m35_evidence_catalog_foundation.md</code>, <code>src/catalog/*.py</code>, <code>src/cli/*.py</code>, <code>src/data/feature_metadata.py</code>, and the M36 test suite. 1 2 12 13 14 15 16 4 5 19 6 17 21 18 22 20 31 29 7 23 24 8 25 26 27 28 3 9 10	Primary evidence on current architecture, code paths, and validation posture
W3C PROV-Overview and PROV-DM. 38	Official provenance model background and provenance-of-provenance semantics
Microsoft Learn, CQRS pattern. 33	Official guidance on single source of truth vs. read models/materialized views
Singh, Cobbe, Norval, <i>Decision Provenance: Harnessing data flow for accountable systems</i> . 39	Why provenance matters for accountability, audit, and oversight
Leo et al., <i>Recording provenance of workflow runs with RO-Crate</i> . 40	Interoperable packaging of workflow provenance and associated objects
Pasquier et al., <i>Practical Whole-System Provenance Capture</i> . 35	Provenance volume, tailoring, and capture/consumption trade-offs

1 **README.md**

<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/README.md>

2 **m36_release_notes.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m36_release_notes.md

3 **m36_release_validation_checklist.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m36_release_validation_checklist.md

4 **derived_index.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/catalog/derived_index.py

5 **lineage_export.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/catalog/lineage_export.py

6 **workflows.py**

<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/catalog/workflows.py>

7 **test_m36_deterministic_validation.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/tests/test_m36_deterministic_validation.py

8 **test_catalog_cli_api_ergonomics.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/tests/test_catalog_cli_api_ergonomics.py

9 **milestone_29_unified_research_catalog_design.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/milestone_29_unified_research_catalog_design.md

10 **m35_evidence_catalog_foundation.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/m35_evidence_catalog_foundation.md

11 30 34 38 **<https://www.w3.org/TR/prov-overview/>**

<https://www.w3.org/TR/prov-overview/>

12 **catalog_indexer.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/catalog_indexer.md

13 **catalog_lineage.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/catalog_lineage.md

14 **catalog_query.md**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/docs/catalog_query.md

15 **__init__.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/catalog/__init__.py

16 **models.py**

<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/catalog/models.py>

17 **catalog_index.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/cli/catalog_index.py

18 **export_catalog_lineage.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/cli/export_catalog_lineage.py

19 **lineage_fingerprints.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/catalog/lineage_fingerprints.py

20 **feature_metadata.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/data/feature_metadata.py

21 **query_catalog.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/cli/query_catalog.py

22 **explore_catalog_evidence.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/cli/explore_catalog_evidence.py

23 **test_catalog_lineage_export.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/tests/test_catalog_lineage_export.py

24 **test_dataset_feature_lineage.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/tests/test_dataset_feature_lineage.py

25 **catalog_scale_fixtures.py**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/tests/catalog_scale_fixtures.py

26 **milestone_validation.yml**

https://github.com/christophermoverton/stratlake-trade-engine/blob/main/.github/workflows/milestone_validation.yml

27 **release.yml**

<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/.github/workflows/release.yml>

28 **ci.yml**

<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/.github/workflows/ci.yml>

31 **indexer.py**

<https://github.com/christophermoverton/stratlake-trade-engine/blob/main/src/catalog/indexer.py>

32 36 37 39 **<https://arxiv.org/abs/1804.05741>**

<https://arxiv.org/abs/1804.05741>

33 **<https://docs.microsoft.com/en-us/azure/architecture/patterns/cqrs>**

<https://docs.microsoft.com/en-us/azure/architecture/patterns/cqrs>

35 **<https://arxiv.org/abs/1711.05296>**

<https://arxiv.org/abs/1711.05296>

40 **<https://arxiv.org/abs/2312.07852>**

<https://arxiv.org/abs/2312.07852>